

Full Stack Developer Interview Assignment

Preferred stack	Django, React, SQL
Nature of role	Backend-heavy full stack development
Domain context	Field Force Management System with role-based and multi-level access control
Expected completion time	3 days

Overview

Please complete one of the two assignments in this document. These assignments are designed around a Field Force Management System where role-based access control, module-level permissions, and scoped data visibility are essential. The primary focus is on backend architecture, API design, access control, database modeling, and practical SQL thinking.

General Instructions

- Submit the solution through a GitHub repository.
- Include a proper README with project overview, setup instructions, dependency installation steps, database migration steps, how to run backend locally, how to run frontend locally, sample credentials or seed data instructions, and assumptions or tradeoffs.
- Running locally is sufficient. AWS and deployment are not required.
- UI polish is not the priority. Functional correctness, backend quality, and clarity of implementation matter more.

Note : You will have 3 days from the date of assignment to submit your work; we encourage you to prioritize effectively and deliver the best possible outcome within this timeframe, as submissions will not be accepted after the deadline.

AI Requirement

The AI feature must be implemented as a mocked module only. Candidates should not use paid AI APIs, external AI services, or real LLM integrations. The purpose is to evaluate how the AI-like feature is designed and integrated into the product flow, not whether a real provider is used.

- Examples of acceptable mocked AI behavior include generating a short summary from notes, returning a follow-up recommendation using simple rules, or assigning a deterministic risk or priority flag.
- The mocked AI logic should ideally be placed behind a clean service abstraction so it can later be replaced by a real AI integration.

Assignment Option 1: Field Operations Task and Visit Management

Problem Statement

Build a mini field operations system where managers assign tasks to field agents, agents perform visits, and users can access data and actions only according to their role and allowed scope. The solution should reflect realistic operational workflows in a field force environment.

Functional Requirements

1. Authentication and User Roles

- Implement login or authentication and role-based access control.
- Suggested roles: Admin, Regional Manager, Team Lead, Field Agent, and Auditor.
- Example behavior: Admin can manage everything, Regional Manager can manage data in scope, Team Lead can monitor and assign work to team members, Field Agent can update only their own visits, and Auditor can view reports and logs only.

2. Core Modules

- Task Management: create tasks, assign tasks, update task status, and list tasks according to user access.
- Visit Tracking: start a visit, complete a visit, add visit notes, and update visit outcome or status.
- Scope-Based Visibility: visibility should depend on role, team or region or zone, module permissions, and action permissions where relevant.
- Activity Logging: track important actions such as task creation, assignment, visit started, visit completed, and status changed.

3. Mock AI Integration

- When visit notes are submitted, generate one or more mocked AI outputs such as a visit summary, follow-up recommendation, risk flag, or suggested next action.
- Store both the original visit notes and the mocked AI output.
- A separate service or module for mocked AI behavior is preferred.

Suggested Entities

- User
- Role
- Permission or ModuleAccess
- Team, Region, or Zone
- EmployeeProfile
- Task
- Visit
- ActivityLog

Backend and API Expectations

- Login or authentication
- Create task
- Assign task
- List tasks
- Get task details
- Start visit
- Complete visit

- Add visit notes
- Fetch dashboard or summary data
- Fetch logs or report data based on access

Frontend Expectations

- Login page
- Dashboard
- Task list page
- Task detail page
- Visit update form
- Role-aware navigation or sidebar
- Mock AI summary display section

SQL and Reporting Expectations

- Include at least three meaningful reporting or query use cases such as pending tasks by region or team, average completion time per field agent, visits completed in the last seven days, or task status distribution by manager or team.

Good-to-Have (Optional)

- JWT authentication
- Search, filtering, or pagination
- Test cases for permission logic
- Better audit trail handling

Assignment Option 2: Lead Allocation and Escalation System

Problem Statement

Build a backend-heavy lead management workflow for a Field Force Management System where incoming leads are assigned to field users, activities are tracked, escalations happen based on rules, and access is controlled through role-based and hierarchy-based permissions. This option is intended to test deeper ownership-based visibility, reassignment logic, escalation handling, and backend maturity.

Functional Requirements

1. Authentication and Role-Based Access

- Implement login or authentication and RBAC.
- Suggested roles: Admin, National Manager, Area Manager, Team Lead, Field Executive, and Operations Reviewer.
- Example behavior: Admin can manage all data, National Manager can view all leads, Area Manager can manage leads in territory, Team Lead can manage team-level leads, Field Executive can access only their assigned leads, and Operations Reviewer can view reports with limited update access.

2. Core Modules

- Lead Management: create lead, assign lead, reassign lead, update lead status, add lead activity notes, and view leads according to access.

- Escalation Workflow: escalate leads if no action is taken for a period of time, restrict who can close or reassign escalated leads, and restrict reassignment outside allowed scope.
- Hierarchical or Scoped Visibility: visibility should depend on role, ownership, reporting hierarchy, territory or team or region, and module permissions.
- Activity Tracking: track lead created, assigned, reassigned, escalated, and status changed actions.

3. Mock AI Integration

- Add one useful mocked AI feature such as classifying lead priority, suggesting next best action, or summarizing recent lead activity.
- Store both the source data used for the mocked AI output and the generated mocked output.
- A clean service abstraction is encouraged.

Suggested Entities

- User
- Role
- ModulePermission
- ReportingHierarchy
- Territory, Team, or Region
- Lead
- LeadActivity
- EscalationRule
- AssignmentRule

Backend and API Expectations

- Login or authentication
- Create lead
- Assign lead
- Reassign lead
- Update lead status
- Escalate lead
- List leads visible to current user
- Get lead details
- Fetch dashboard or report data

Frontend Expectations

- Login page
- Dashboard
- Leads list page
- Lead details page
- Assignment or reassignment flow
- Escalation action flow
- Permission-aware navigation or modules

SQL and Reporting Expectations

- Include at least three meaningful reporting or query use cases such as leads aging report, uncontacted leads by territory, escalation count by manager or team, conversion by assignee or source, or top performers by closure rate.

Good-to-Have (Optional)

- Bulk CSV import
- Reusable permission middleware or decorators
- Unit tests for access control
- Swagger or OpenAPI docs
- Seed scripts for sample data

Submission Checklist

Item	Details
GitHub repository	Complete source code must be shared via a GitHub repository.
README	Include setup, dependency installation, database migration steps, how to run backend locally, how to run frontend locally, sample credentials or seed instructions, and assumptions/tradeoffs.
Database artifacts	Migration files and any seed or dummy data instructions.
API reference	Postman collection or brief API documentation.
Scope note	Running locally is sufficient. AWS and deployment are not required.

Evaluation Rubric

Area	Weight	What We Will Look For
Backend architecture	25%	Project structure, separation of concerns, maintainability, and API design quality.
Access control	25%	Correct RBAC, multi-level visibility, module restrictions, and safe data access.
Database and SQL	20%	Schema quality, relationships, reporting queries, and practical relational thinking.
Frontend integration	10%	Working React flow, correct API usage, and permission-aware rendering.
Mock AI design	10%	Clean abstraction of mocked AI logic and sensible workflow integration.

Documentation	10%	Readable README, local setup clarity, and complete submission quality.
---------------	-----	--